

Production-Grade Multi-Agent Agricultural Advisory System: Backend Architecture and Implementation

The deployment of artificial intelligence within the government technology (GovTech) sector requires architectures that balance the probabilistic flexibility of Large Language Models (LLMs) with the strict, deterministic boundaries of public policy. The following report details the comprehensive backend implementation of a production-grade Agricultural Advisory System. This system leverages a multi-agent architecture powered by LangGraph, utilizing Sarvam AI for highly accurate, localized voice processing, and a hybrid eligibility engine to facilitate automated scheme discovery and form fulfillment. The architecture addresses the critical requirements of offline-first potential, multi-lingual support, and verifiable data collection.

Part 1: Scheme Data Sources and Ingestion Pipeline

The foundation of any advisory agent is its knowledge base. To ensure high-fidelity responses, the system must continually ingest, structure, and update data from authoritative government platforms. The fragmented landscape of Indian agricultural data requires integration across central platforms like the myScheme portal, India.gov.in, the PM-Kisan official site, and state-level portals such as Maharashtra's MahaDBT.¹

Data Access and Scraping Strategies

The Indian government has increasingly adopted Open API policies through platforms like API Setu, which facilitates interoperable data exchange across e-Governance silos.⁴ The myScheme portal and various agricultural datasets are accessible via API Setu, providing structured JSON payloads containing scheme metadata.⁵ However, for portals lacking robust API endpoints, such as specific state-level agricultural mechanization schemes or localized subsidy programs, headless browser automation (e.g., Selenium or Playwright) combined with asynchronous HTTP clients must be deployed to extract DOM elements.⁷

The data extracted from these diverse sources must be standardized into a unified schema to facilitate deterministic filtering and semantic search. Table 1 illustrates the normalized structure required for the system to process both central schemes (like PM-Kisan) and state schemes (like Maharashtra's Tractor Subsidy).

Field Name	Data Type	Description	Example (PM-Kisan)

scheme_id	String	Unique identifier for the scheme.	PM-KISAN-001
name	String	Official title of the government scheme.	Pradhan Mantri Kisan Samman Nidhi
state	String	Geographic applicability (Central or State name).	Central
benefits	Dictionary	Structured representation of financial or material aid.	{"amount": 6000, "frequency": "Annual"}
eligibility_criteria	Dictionary	Nested conditions for programmatic evaluation.	{"max_land_size_hectares": 2.0, "farmer_category":{}}
required_fields	Array	Data points the agent must collect from the user.	["farmer_name", "aadhaar_number", "land_area"]
documents_required	Array	Physical or digital proofs required for submission.	``

Table 1: Normalized JSON Schema for Government Agricultural Schemes.

Asynchronous Data Ingestion Code

The following implementation utilizes Python's asynchronous capabilities to pull data, transform

it to match the unified schema, and store it within a PostgreSQL database using SQLAlchemy. This module ensures that the knowledge base remains synchronized with upstream government portals.

Python

```
# services/scheme_service.py
import asyncio
import json
import logging
from typing import List, Dict, Any, Optional
import aiohttp
from bs4 import BeautifulSoup
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
from sqlalchemy import text

logger = logging.getLogger(__name__)

class SchemeIngestionPipeline:
    """
    Handles the extraction, normalization, and storage of agricultural schemes
    from API Setu and direct portal scraping.
    """
    def __init__(self, db_url: str, api_setu_key: str):
        self.engine = create_async_engine(db_url, echo=False, pool_size=20, max_overflow=10)
        self.async_session = sessionmaker(self.engine, class_=AsyncSession,
        expire_on_commit=False)
        self.api_setu_key = api_setu_key
        self.headers = {
            "Authorization": f"Bearer {self.api_setu_key}",
            "Accept": "application/json"
        }

    async def fetch_api_setu_schemes(self) -> List]:
        """Fetches structured scheme data from API Setu endpoints."""
        url = "https://apisetu.gov.in/api/v1/schemes/agriculture"
        async with aiohttp.ClientSession(headers=self.headers) as session:
            try:
                async with session.get(url, timeout=30) as response:
```

```

        if response.status == 200:
            raw_data = await response.json()
            return self._normalize_api_setu_data(raw_data)
        else:
            logger.error(f"API Setu returned status {response.status}")
            return self._fallback_local_data()
    except asyncio.TimeoutError:
        logger.error("API Setu request timed out.")
        return self._fallback_local_data()
    except Exception as e:
        logger.error(f"Ingestion failed: {str(e)}")
        return self._fallback_local_data()

async def scrape_mahadbt_schemes(self) -> List]:
    """Scrapes unstructured data from Maharashtra state portals."""
    # Simulated scraping logic for state-level portals
    url = "https://mahadbt.maharashtra.gov.in/Farmer/SchemeData"
    async with aiohttp.ClientSession() as session:
        try:
            async with session.get(url) as response:
                html = await response.text()
                soup = BeautifulSoup(html, 'html.parser')
                # Extraction logic tailored to the specific DOM structure
                # Example: extracting the State Agriculture Mechanization Scheme
                return,
                    "exclusions": ["Prior beneficiary in last 5 years"]
                },
                "required_fields": ["farmer_name", "aadhaar_number", "caste_category"],
                "documents_required":
            ]]
        except Exception as e:
            logger.error(f"Scraping failed: {str(e)}")
        return

def _normalize_api_setu_data(self, raw_data: List] -> List]:
    """Transforms external API schemas into the internal standardized JSON structure."""
    normalized =
    for item in raw_data:
        normalized.append({
            "scheme_id": item.get("id", f"SCHEME-{uuid.uuid4().hex[:8]}"),
            "name": item.get("scheme_name", "Unknown Scheme"),
            "state": item.get("state", "Central"),
            "description": item.get("details", ""),

```

```

        "benefits": item.get("financial_benefits", {}),
        "eligibility_criteria": item.get("eligibility", {}),
        "required_fields": item.get("required_parameters"),
        "documents_required": item.get("documents"),
    })
    return normalized

```

```

def _fallback_local_data(self) -> List:
    """Fallback to highly verified local JSON if upstream APIs fail."""
    return,
        "exclusions":
    },
    "required_fields": ["farmer_name", "aadhaar_number", "bank_account", "ifsc_code", "land_area"],
    "documents_required":
    ]]

```

```

async def store_scheme_metadata(self, schemes: List):
    """Upserts the raw JSON schemas into PostgreSQL for downstream retrieval."""
    insert_query = text("""
        INSERT INTO scheme_metadata (scheme_id, state, data, updated_at)
        VALUES (:scheme_id, :state, :data, CURRENT_TIMESTAMP)
        ON CONFLICT (scheme_id) DO UPDATE SET
            data = EXCLUDED.data,
            updated_at = CURRENT_TIMESTAMP
    """)
    async with self.async_session() as session:
        for scheme in schemes:
            await session.execute(insert_query, {
                "scheme_id": scheme["scheme_id"],
                "state": scheme["state"],
                "data": json.dumps(scheme)
            })
        await session.commit()
    logger.info(f"Successfully stored {len(schemes)} schemes in relational storage.")

```

```

async def execute_pipeline(self):
    """Orchestrates the extraction and storage processes."""
    api_data = await self.fetch_api_setu_schemes()
    scraped_data = await self.scrape_mahadbt_schemes()
    all_schemes = api_data + scraped_data
    await self.store_scheme_metadata(all_schemes)
    return all_schemes

```

Part 2: RAG and Vector Retrieval System

To ensure the LangGraph agent can dynamically retrieve relevant schemes based on a user's conversational input, a Retrieval-Augmented Generation (RAG) system is strictly necessary. For GovTech applications, exact metadata filtering is equally as important as semantic similarity.⁸ If a farmer resides in Maharashtra, retrieving a subsidy scheme exclusive to Punjab—regardless of semantic similarity—is a critical system failure.

PostgreSQL, augmented with the pgvector extension, provides a robust infrastructure for combining exact relational filtering with highly dimensional vector similarity search.⁸ This dual-capability ensures that the vector search space is pre-filtered by hard demographic realities before the distance metric (e.g., cosine similarity) is calculated.

Embedding Generation and Vector Storage Implementation

The retrieval service chunks the comprehensive JSON data into semantic blocks, generates embeddings, and executes filtered similarity searches.

Python

```
# services/retrieval_service.py
import json
import logging
from typing import List, Dict, Any, Tuple
from langchain_openai import OpenAIEmbeddings
from langchain_postgres import PGVector
from langchain_core.documents import Document

logger = logging.getLogger(__name__)

class SchemeRetriever:
    """
    Manages vector embeddings and execution of hybrid similarity searches
    using pgvector.
    """
    def __init__(self, connection_string: str):
        # Utilizing high-dimension embeddings for nuanced semantic matching
        self.embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
        self.collection_name = "agriculture_schemes"

    # Initialize PGVector connection
```

```

self.vector_store = PGVector(
    embeddings=self.embeddings,
    collection_name=self.collection_name,
    connection=connection_string,
    use_jsonb=True,
)

async def ingest_documents(self, schemes: List):
    """Converts scheme JSON into vectorized documents with JSONB metadata."""
    docs =
    for scheme in schemes:
        # Construct a dense text representation to maximize embedding quality
        content = (
            f"Scheme Name: {scheme['name']}. "
            f"State: {scheme['state']}. "
            f"Description: {scheme['description']} "
            f"Benefits include Rs. {scheme.get('benefits', {}).get('amount', 0)} "
            f"paid on a {scheme.get('benefits', {}).get('frequency', 'regular')} basis. "
            f"Eligible Land Size is up to {scheme.get('eligibility_criteria', {}).get('max_land_size_hectares',
'Any')} hectares. "
            f"Targeting farmers in categories: {', '.join(scheme.get('eligibility_criteria',
{}).get('farmer_category', ''))}."
        )

        # Metadata is critical for exact pre-filtering before HNSW/IVFFlat index traversal
        doc = Document(
            id=scheme["scheme_id"],
            page_content=content,
            metadata={
                "state": scheme["state"],
                "max_land_size": scheme.get('eligibility_criteria', {}).get('max_land_size_hectares', 999.0),
                "scheme_id": scheme["scheme_id"],
                "required_fields": scheme.get("required_fields",)
            }
        )
        docs.append(doc)

    try:
        await self.vector_store.aadd_documents(docs)
        logger.info(f"Vectorized and stored {len(docs)} scheme documents.")
    except Exception as e:
        logger.error(f"Vector ingestion failed: {str(e)}")

```

```

async def retrieve_relevant_schemes(
    self,
    query: str,
    user_state: str,
    user_land_size: float = 0.0,
    k: int = 3
) -> List:
    """
    Performs a similarity search strictly gated by metadata filters to prevent
    cross-state hallucination and enforce baseline physical constraints.
    """
    # The $and operator ensures both conditions must be met
    filters = {
        "$and": {},
        # The scheme's max land limit must be greater than or equal to the user's land size
        {"max_land_size": {"$gte": user_land_size}}
    ]

    try:
        # similarity_search_with_score returns Tuple
        results_with_scores = await self.vector_store.asimilarity_search_with_score(
            query=query,
            k=k,
            filter=filters
        )

        # Log scores for observability and thresholding
        valid_docs =
        for doc, score in results_with_scores:
            logger.debug(f"Retrieved {doc.metadata['scheme_id']} with distance {score}")
            # Optional: Filter out results with high distance (low similarity)
            if score < 0.7:
                valid_docs.append(doc)

        return valid_docs

    except Exception as e:
        logger.error(f"Vector retrieval failed: {str(e)}")
        return

```

Part 3: The Hybrid Eligibility Engine

Relying exclusively on generative LLMs for determining eligibility introduces unacceptable risks

of hallucination, non-determinism, and non-compliance with strict government guidelines.¹⁰ A purely rule-based system, conversely, struggles with the unstructured, fuzzy nature of conversational inputs (e.g., a farmer stating, "I have about three acres near the river").¹²

The definitive solution is a **Hybrid Eligibility Engine** following the "Rules-First, LLM-Fallback" pattern.¹⁰ Deterministic rules execute instantaneously and provide auditable guarantees. When an input fails to match predefined structural criteria, the system defers to the LLM to interpret the semantic intent before updating the state.¹⁰

Architectural Pattern and Execution Flow

1. **Pattern Matching Phase:** Incoming user data is matched against strict regex, numerical thresholds, and dictionary lookups.¹⁰
2. **Deterministic Evaluation:** If the data satisfies the rule completely, the engine immediately yields a decision, bypassing the LLM entirely (saving latency and token costs).
3. **Semantic Fallback:** If the rule evaluation encounters ambiguity (e.g., unparseable text formats for land area), a `skip_llm` flag is toggled to `False`, invoking the LLM to contextually deduce eligibility.¹⁰

Code Implementation

Python

```
# services/eligibility_engine.py
import json
import logging
from typing import Dict, Any, Tuple, List
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

logger = logging.getLogger(__name__)

class HybridEligibilityEngine:
    """
    Evaluates farmer eligibility using a deterministic forward-chaining rule
    evaluation, falling back to an LLM for fuzzy logic resolution.
    """
    def __init__(self):
        # Temperature 0 is critical for predictable JSON generation in fallback
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

```

def _evaluate_deterministic_rules(
    self,
    user_data: Dict[str, Any],
    scheme_rules: Dict[str, Any]
) -> Tuple[bool, bool, List[str], str, float]:
    """
    Evaluates hard business logic based on defined JSON schemas.
    Returns: (handled_by_rules, is_eligible, missing_fields, reason, confidence_score)
    """
    missing_fields = []
    required_fields = scheme_rules.get("required_fields", [])

    # 1. Verification of Required Fields
    for field in required_fields:
        if field not in user_data or user_data[field] in [None, "", "unknown"]:
            missing_fields.append(field)

    if missing_fields:
        # Deterministic failure due to missing data. High confidence.
        return True, False, missing_fields, f"Missing required fields: {', '.join(missing_fields)}", 1.0

    # 2. Numeric Threshold Evaluation (e.g., Land Size)
    max_land = scheme_rules.get("eligibility_criteria", {}).get("max_land_size_hectares")
    if max_land and "land_area" in user_data:
        try:
            # Attempt strict type conversion
            user_land = float(user_data["land_area"])
            if user_land > max_land:
                return True, False, f"Land size {user_land} ha exceeds limit of {max_land} ha.", 1.0
        except (ValueError, TypeError):
            # Value is fuzzy (e.g., "around 2 acres"). Hand control to LLM fallback.
            return False, False, "", 0.0

    # 3. Categorical Exclusions (e.g., Pensioners, Institutional Landholders)
    exclusions = scheme_rules.get("eligibility_criteria", {}).get("exclusions", [])
    user_category = user_data.get("farmer_category", "")
    if user_category in exclusions:
        return True, False, f"Ineligible: User falls under exclusion category '{user_category}'.", 1.0

    # If data is perfectly structured and clears all hurdles
    return True, True, "User meets all deterministic criteria.", 1.0

```

```

async def _evaluate_llm_fallback(
    self,
    user_data: Dict[str, Any],
    scheme_rules: Dict[str, Any]
) -> Dict[str, Any]:
    """
    Uses an LLM to interpret fuzzy conditions, acting as the cognitive fallback.
    """

    prompt = PromptTemplate.from_template("""
    You are an expert government compliance officer evaluating agricultural scheme eligibility.
    Evaluate the following User Data against the Scheme Rules.
    Convert fuzzy regional measurements (like acres or bighas) to hectares if necessary to evaluate
    rules.
    1 Hectare = 2.47 Acres.

    Scheme Rules: {rules}
    User Data: {user_data}

    Provide a confidence score between 0.0 and 1.0 based on how clear the user's data is.
    Return ONLY a valid JSON object matching this exact schema:
    {{
        "eligible": boolean,
        "missing_fields": ["array", "of", "missing", "keys"],
        "reason": "Detailed explanation of the decision",
        "confidence_score": float
    }}
    """)

    chain = prompt | self.llm
    response = await chain.ainvoke({
        "rules": json.dumps(scheme_rules),
        "user_data": json.dumps(user_data)
    })

    try:
        # Strip markdown formatting often returned by generative models
        raw_json = response.content.replace("```json", "").replace("```", "").strip()
        result = json.loads(raw_json)
        return result
    except json.JSONDecodeError as e:
        logger.error(f"LLM generated invalid JSON: {str(e)}. Content: {response.content}")
        return {
            "eligible": False,
            "missing_fields":,
            "reason": "System error during semantic eligibility parsing.",

```

```

        "confidence_score": 0.0
    }

    async def check_eligibility(
        self,
        user_data: Dict[str, Any],
        scheme_rules: Dict[str, Any]
    ) -> Dict[str, Any]:
        """Main entry point for the hybrid engine."""
        handled, eligible, missing, reason, confidence =
self._evaluate_deterministic_rules(user_data, scheme_rules)

        if handled:
            logger.info("Eligibility conclusively handled by deterministic rule engine.")
            return {
                "eligible": eligible,
                "missing_fields": missing,
                "reason": reason,
                "confidence_score": confidence
            }

        logger.info("Fuzzy data detected. Falling back to LLM semantic evaluation.")
        return await self._evaluate_llm_fallback(user_data, scheme_rules)

```

Part 4: Voice-First Interaction Pipeline with Sarvam AI

To serve the rural agricultural demographic effectively, the system must accommodate regional dialects, ambient noise, and code-mixing (e.g., Hindi interspersed with English technical terms). Sarvam AI's saaras:v3 Speech-to-Text (STT) and bulbul:v3 Text-to-Speech (TTS) APIs are uniquely optimized for these Indian linguistic nuances, outperforming generic models on localized phonetics.¹³

The voice service acts as the primary sensory input/output layer for the LangGraph agent. It manages asynchronous HTTP requests to Sarvam's REST endpoints, converting raw audio bytes to text, and synthesizing the agent's textual responses back into high-fidelity audio arrays.¹⁵

STT, Extraction, and TTS Implementation

Python

```
# services/voice_service.py
```

```
import aiohttp
import base64
import logging
from typing import Optional
```

```
logger = logging.getLogger(__name__)
```

```
class SarvamVoiceService:
```

```
    """
```

```
    Integrates Sarvam AI endpoints for Indic language Speech-to-Text
    and Text-to-Speech processing.
```

```
    """
```

```
    def __init__(self, api_key: str):
        self.api_key = api_key
        self.stt_url = "https://api.sarvam.ai/speech-to-text"
        self.tts_url = "https://api.sarvam.ai/text-to-speech"
        self.headers = {
            "api-subscription-key": self.api_key,
            "Content-Type": "application/json"
        }
```

```
    async def speech_to_text(self, audio_bytes: bytes, language_code: str = "hi-IN") -> str:
```

```
        """
```

```
        Transcribes incoming audio using Saaras v3.
        The model inherently handles code-mixing and background noise.
```

```
        """
```

```
        # Aiohttp requires FormData for multipart/form-data uploads
        form_data = aiohttp.FormData()
        form_data.add_field('file', audio_bytes, filename='input_audio.wav', content_type='audio/wav')
        form_data.add_field('model', 'saaras:v3')
        form_data.add_field('mode', 'transcribe')
```

```
        # Override headers; multipart sets its own boundary/content-type
        stt_headers = {"api-subscription-key": self.api_key}
```

```
    async with aiohttp.ClientSession(headers=stt_headers) as session:
```

```
        try:
```

```
            async with session.post(self.stt_url, data=form_data, timeout=15) as response:
```

```
                if response.status == 200:
```

```
                    data = await response.json()
```

```
                    transcript = data.get("transcript", "").strip()
```

```
                    logger.info(f"STT Transcript generated successfully.")
```

```
                    return transcript
```

```

        else:
            error_text = await response.text()
            logger.error(f"Sarvam STT HTTP Error {response.status}: {error_text}")
            return ""
    except Exception as e:
        logger.error(f"STT Network Exception: {str(e)}")
        return ""

    async def text_to_speech(
        self,
        text: str,
        target_language: str = "hi-IN",
        speaker: str = "shubh"
    ) -> Optional[bytes]:
        """
        Synthesizes text to speech using Bulbul v3.
        Supports granular controls over pitch, pace, and sample rate.
        """
        payload = {
            "inputs": [text],
            "target_language_code": target_language,
            "speaker": speaker,
            "pitch": 0,
            "pace": 1.1, # Slightly faster pace for conversational cadence
            "loudness": 1.2,
            "speech_sample_rate": 24000,
            "model": "bulbul:v3"
        }

    async with aiohttp.ClientSession(headers=self.headers) as session:
        try:
            async with session.post(self.tts_url, json=payload, timeout=10) as response:
                if response.status == 200:
                    data = await response.json()
                    # Sarvam returns a list of base64 encoded audio strings corresponding to inputs
                    audio_b64 = data.get("audios",)
                    return base64.b64decode(audio_b64)
                else:
                    error_text = await response.text()
                    logger.error(f"Sarvam TTS HTTP Error {response.status}: {error_text}")
                    return None
        except Exception as e:
            logger.error(f"TTS Network Exception: {str(e)}")
            return None

```

Part 5: LangGraph Conversational Data Collection Engine

The crux of the multi-agent system resides in conversational orchestration. While standard prompt-chaining is susceptible to derailment, LangGraph utilizes a state machine paradigm, enabling durable execution, cyclical flows, and precise control over conversational routing.¹⁶

The state machine manages an AgentState object containing the missing_fields list. Rather than overwhelming the user by requesting all missing information simultaneously, the graph iteratively loops through a next_question node, prompting the user for exactly one parameter per turn until the state is fully resolved.¹⁸

State Definition, Nodes, and Conditional Routing

Python

```
# agents/scheme_agent.py
import operator
import json
from typing import TypedDict, Annotated, List, Dict, Any, Sequence
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langgraph.graph import StateGraph, END
from services.eligibility_engine import HybridEligibilityEngine

# 1. Define the persistent Graph State
class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], operator.add]
    user_profile: Dict[str, Any] # Base demographic data
    collected_data: Dict[str, Any]
    missing_fields: List[str]
    target_scheme: Dict[str, Any]
    current_step: str
    is_eligible: bool
    retry_count: int
    form_pdf_path: str
    language_pref: str # e.g., "hi-IN" or "mr-IN"
```

```
engine = HybridEligibilityEngine()
llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

```
# 2. Node: Entity Extraction via LLM Tool Calling
```

```
async def extract_entities_node(state: AgentState) -> Dict[str, Any]:
```

```
    """
```

```
    Analyzes the latest HumanMessage and extracts entities to populate collected_data.
    Maintains a retry_count to handle continuous failures gracefully.
```

```
    """
```

```
    last_message = state["messages"][-1].content
```

```
    extraction_prompt = PromptTemplate.from_template("""
```

```
        Extract any agricultural or identity information from the text.
        Text: {text}
    """)
```

```
    Return a valid JSON object with keys like "aadhaar_number", "land_area", "bank_account", etc.
    If no relevant entities are found, return an empty dictionary {}.
```

```
    """)
```

```
    chain = extraction_prompt | llm
```

```
    response = await chain.ainvoke({"text": last_message})
```

```
    try:
```

```
        raw_json = response.content.replace("```json", "").replace("```", "").strip()
```

```
        new_entities = json.loads(raw_json)
```

```
        updated_data = state["collected_data"].copy()
```

```
        updated_data.update(new_entities)
```

```
        # Reset retry count if new data is successfully extracted
```

```
        retry_val = 0 if new_entities else state.get("retry_count", 0) + 1
```

```
        return {"collected_data": updated_data, "retry_count": retry_val}
```

```
    except Exception:
```

```
        return {"retry_count": state.get("retry_count", 0) + 1}
```

```
# 3. Node: Eligibility and Missing Field Check
```

```
async def eligibility_check_node(state: AgentState) -> Dict[str, Any]:
```

```
    """Passes current state to the Hybrid Engine to assess status and missing parameters."""
```

```
    result = await engine.check_eligibility(state["collected_data"], state["target_scheme"])
```

```
    current_step = "eligibility_checked" if not result["missing_fields"] else "collecting_data"
```

```
    if not result["eligible"] and not result["missing_fields"]:
```

```
        current_step = "rejected"
```

```

return {
    "is_eligible": result["eligible"],
    "missing_fields": result["missing_fields"],
    "current_step": current_step
}

```

4. Node: Conversational Flow Generation

```

async def conversational_prompt_node(state: AgentState) -> Dict[str, Any]:
    """Generates the next question systematically based on the missing_fields queue."""
    if state.get("retry_count", 0) > 3:
        return {"messages":}

    missing = state["missing_fields"]
    language = state.get("language_pref", "hi-IN")

    if missing:
        next_field = missing
        # Multi-language translation matrix for standard required fields
        prompts = {
            "hi-IN": {
                "aadhaar_number": "योजना के लिए आपका बारह अंकों का आधार नंबर आवश्यक है। कृपया अपना आधार नंबर बताएं।",
                "bank_account": "सब्सिडी के लिए आपके बैंक खाते का नंबर क्या है?",
                "land_area": "आपकी खेती की जमीन कितने हेक्टेयर है?"
            },
            "mr-IN": {
                "aadhaar_number": "योजनेसाठी तुमचा बारा अंकी आधार क्रमांक आवश्यक आहे. कृपया तुमचा आधार क्रमांक सांगा.",
                "bank_account": "अनुदानासाठी तुमचा बँक खाते क्रमांक काय आहे?",
                "land_area": "तुमची शेतजमीन किती हेक्टर आहे?"
            }
        }

        # Default to dynamic LLM generation if the field is obscure
        fallback_msg = f"Please provide your {next_field.replace('_', '')}."
        question = prompts.get(language, {}).get(next_field, fallback_msg)

        return {"messages": [AIMessage(content=question)]}

    return {"messages": [AIMessage(content="All necessary data has been collected. I am preparing your application form now.")]}

```

5. Define Edge Routing Logic

```
def routing_logic(state: AgentState) -> str:
```

```
    """Evaluates the state after eligibility check to route the graph."""
```

```
    if state["current_step"] == "rejected":
```

```
        return "rejection_end"
```

```
    if state["missing_fields"]:
```

```
        return "ask_question"
```

```
    return "generate_form"
```

Part 6: Automated Form Generation with PyMuPDF

Once the conversational engine resolves all missing variables and confirms eligibility, the pipeline transitions to the physical form fulfillment stage. Official templates (e.g., the standard PM-Kisan offline application) are programmatically filled mapping the populated JSON state dictionary to the PDF widgets.

PyMuPDF (fitz) is utilized because of its high-performance widget modification capabilities and its ability to robustly "flatten" interactive PDFs.¹⁹ Flattening is a critical security requirement; by rendering the document into static images embedded in a new PDF shell, post-generation tampering of banking or identification details is rendered impossible.²⁰

Python

```
# services/form_filler.py
```

```
import fitz # PyMuPDF
```

```
import logging
```

```
import os
```

```
from typing import Dict, Any
```

```
logger = logging.getLogger(__name__)
```

```
class PDFFormFiller:
```

```
    """
```

```
    Automates the insertion of user state dictionaries into standardized
```

```
    AcroForm PDF templates, securing the output via flattening.
```

```
    """
```

```
    def __init__(self, template_dir: str = "templates/"): 
```

```
        self.template_dir = template_dir
```

```
    def generate_application_form(self, user_data: Dict[str, Any], scheme_id: str, output_path: str) -> str:
```

```
        """
```

Maps JSON data to PDF widgets, updates values, and flattens the document.

"""

```
template_path = os.path.join(self.template_dir, f"{scheme_id}_template.pdf")
```

```
if not os.path.exists(template_path):
```

```
    logger.error(f"Template for {scheme_id} not found at {template_path}")
```

```
    raise FileNotFoundError(f"Missing template for {scheme_id}")
```

```
try:
```

```
    # 1. Open the interactive PDF template
```

```
    doc = fitz.open(template_path)
```

```
    # Field mapping protocol: Maps PDF AcroForm Widget Names to internal JSON Keys
```

```
    # In a full deployment, this mapping resides in the database per scheme
```

```
    field_mapping = {
```

```
        "Text_FarmerName": "farmer_name",
```

```
        "Text_Aadhaar": "aadhaar_number",
```

```
        "Text_BankAcc": "bank_account",
```

```
        "Text_IFSC": "ifsc_code",
```

```
        "Text_LandArea": "land_area",
```

```
        "Dropdown_Category": "farmer_category"
```

```
    }
```

```
    # 2. Iterate through document pages and extract all interactive widgets
```

```
    for page in doc:
```

```
        widgets = page.widgets()
```

```
        for widget in widgets:
```

```
            field_name = widget.field_name
```

```
            if field_name in field_mapping:
```

```
                json_key = field_mapping[field_name]
```

```
                if json_key in user_data:
```

```
                    # 3. Mutate the widget value
```

```
                    widget.field_value = str(user_data[json_key])
```

```
                    widget.update()
```

```
    # 4. Flattening Protocol: Render the PDF to image arrays to lock fields
```

```
    imaged_doc = fitz.open()
```

```
    for page in doc:
```

```
        # 300 DPI provides standard print-ready resolution
```

```
        pix = page.get_pixmap(dpi=300)
```

```
        img_page = imaged_doc.new_page(width=page.rect.width, height=page.rect.height)
```

```
        img_page.insert_image(page.rect, pixmap=pix)
```

```

# Save the secured, deflated output
imaged_doc.save(output_path, deflate=True)
doc.close()
imaged_doc.close()

logger.info(f"Successfully generated flattened PDF at {output_path}")
return output_path

except Exception as e:
    logger.error(f"Failed to generate form: {str(e)}")
    raise e

```

Part 7: Status Tracking and Human-in-the-Loop Validation

Asynchronous processing workflows spanning multiple interactions require rigorous state tracking. Furthermore, GovTech systems necessitate explicit human validation before submitting critical forms bearing Aadhaar and banking details to government API gateways.

LangGraph natively supports Human-In-The-Loop (HITL) workflows via its dynamic interrupt() method. When invoked, interrupt() pauses the graph's execution, serializes the current state into the persistent Checkpointer, and waits indefinitely until an external system (via an API command) signals it to resume.²¹

HITL Node and FastAPI Endpoints

Table 2 highlights the strict state enums the system maintains to ensure synchronous awareness between the UI, the Voice Interface, and the LangGraph backend.

State String	Definition
scheme_identified	RAG retrieval successful; target schema loaded.
eligibility_checked	Rule engine evaluated base demographics.

collecting_data	System actively iterating missing fields via Voice.
form_generated	All fields satisfied; PyMuPDF completed rendering.
awaiting_user_confirmation	Graph suspended via interrupt(); awaiting POST.
completed	Form submitted; transaction concluded.

Table 2: Defined Workflow Status Indicators.

Python

```
# pipeline/endpoints.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Dict, Any

app = FastAPI()
# Assuming `orchestrator` is a global instance of SchemeOrchestrator (defined in Part 8)

class ConfirmRequest(BaseModel):
    thread_id: str
    approved: bool

@app.get("/status/{thread_id}")
async def get_status(thread_id: str) -> Dict[str, Any]:
    """
    Retrieves the exact execution state of the conversation graph.
    Useful for frontends polling for progress.
    """
    try:
```

```

    config = {"configurable": {"thread_id": thread_id}}
    state_snapshot = orchestrator.checkpointer.get(config)

    if not state_snapshot:
        raise HTTPException(status_code=404, detail="Thread not found")

    # Extract the current state dictionary
    current_state = state_snapshot.values

    return {
        "current_step": current_state.get("current_step"),
        "collected_data": current_state.get("collected_data", {}),
        "missing_fields": current_state.get("missing_fields",),
        "form_pdf_path": current_state.get("form_pdf_path")
    }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/confirm")
async def confirm_submission(req: ConfirmRequest):
    """
    Resumes a LangGraph thread that is paused at an interrupt().
    Injects the user's boolean approval decision.
    """
    config = {"configurable": {"thread_id": req.thread_id}}

    try:
        # Use the Command object to inject data and resume the graph
        from langgraph.types import Command

        command = Command(resume={"approved": req.approved})

        # Invoke the graph with the resume command
        output_state = await orchestrator.app.ainvoke(command, config=config)

        return {
            "status": "resumed",
            "final_step": output_state.get("current_step")
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to resume graph: {str(e)}")

# Corresponding Agent Node (Added to agents/scheme_agent.py)

```

```

from langgraph.types import interrupt, Command

async def human_validation_node(state: AgentState) -> Command:
    """
    Pauses graph execution. Yields control back to the orchestrator
    until the /confirm API endpoint is called.
    """
    approval_request = {
        "status": "awaiting_user_confirmation",
        "form_path": state["form_pdf_path"],
        "data_summary": state["collected_data"]
    }

    # Graph halts execution at this precise line
    user_decision = interrupt(approval_request)

    if user_decision.get("approved") is True:
        # User confirmed -> Route to submission
        return Command(goto="submit_to_portal", update={"current_step": "completed"})
    else:
        # User rejected -> Route back to entity extraction/correction
        return Command(goto="extract_entities", update={"current_step": "collecting_data"})

async def submit_to_portal_node(state: AgentState) -> Dict[str, Any]:
    """Simulates API transmission of the flattened PDF to government endpoints."""
    # Logic to hit API Setu / external endpoints goes here
    return {"messages":}

```

Part 8: Final Pipeline Orchestration

The pipeline module synthesizes the entire architecture. It acts as the central router between the STT/TTS models, the PostgreSQL persistence layer, and the graph execution logic. The `run_scheme_agent` function encapsulates the core interaction loop, abstracting the complexity of the directed cyclic graph from the external application layer.

Python

```

# pipeline/orchestration.py
import uuid
import base64

```

```
import logging
from typing import Dict, Any
```

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from psycopg_pool import AsyncConnectionPool
```

```
from services.voice_service import SarvamVoiceService
from services.form_filler import PDFFormFiller
from services.retrieval_service import SchemeRetriever
from agents.scheme_agent import (
    AgentState, extract_entities_node, eligibility_check_node,
    conversational_prompt_node, human_validation_node,
    submit_to_portal_node, routing_logic
)
```

```
logger = logging.getLogger(__name__)
```

```
class SchemeOrchestrator:
```

```
    """
```

```
    Central pipeline managing the STT -> Graph -> TTS flow.
```

```
    Utilizes AsyncPostgresSaver for durable execution.
```

```
    """
```

```
    def __init__(self, db_connection_string: str, sarvam_key: str):
        self.voice_service = SarvamVoiceService(sarvam_key)
        self.form_filler = PDFFormFiller("templates/")
        self.retriever = SchemeRetriever(db_connection_string)
        self.pool = AsyncConnectionPool(conninfo=db_connection_string, max_size=20)
```

```
        self.app = self._build_graph()
```

```
    def _build_graph(self):
```

```
        """Constructs the LangGraph directed cyclic graph."""
```

```
        workflow = StateGraph(AgentState)
```

```
        # Add Execution Nodes
```

```
        workflow.add_node("extract_entities", extract_entities_node)
```

```
        workflow.add_node("check_eligibility", eligibility_check_node)
```

```
        workflow.add_node("ask_question", conversational_prompt_node)
```

```
        workflow.add_node("generate_form", self._form_generation_wrapper)
```

```
        workflow.add_node("human_validation", human_validation_node)
```

```
        workflow.add_node("submit_to_portal", submit_to_portal_node)
```

```
# Construct Edges and Routing
```

```
workflow.add_edge(START, "extract_entities")
```

```
workflow.add_edge("extract_entities", "check_eligibility")
```

```
# Conditional Router branching based on eligibility and missing fields
```

```
workflow.add_conditional_edges("check_eligibility", routing_logic, {
```

```
    "ask_question": "ask_question",
```

```
    "generate_form": "generate_form",
```

```
    "rejection_end": END
```

```
})
```

```
# When a question is generated, execution ends for the turn (awaiting user response)
```

```
workflow.add_edge("ask_question", END)
```

```
workflow.add_edge("generate_form", "human_validation")
```

```
workflow.add_edge("submit_to_portal", END)
```

```
# Compile with the Checkpointer for state persistence
```

```
checkpointer = AsyncPostgresSaver(self.pool)
```

```
return workflow.compile(checkpointer=checkpointer)
```

```
async def _form_generation_wrapper(self, state: AgentState) -> Dict[str, Any]:
```

```
    """Wrapper node bridging the graph state to the PDF filler service."""
```

```
    scheme_id = state.get("target_scheme", {}).get("scheme_id", "default")
```

```
    output_file = f"outputs/form_{uuid.uuid4().hex[:8]}.pdf"
```

```
    self.form_filler.generate_application_form(
```

```
        user_data=state["collected_data"],
```

```
        scheme_id=scheme_id,
```

```
        output_path=output_file
```

```
    )
```

```
    return {"form_pdf_path": output_file, "current_step": "form_generated"}
```

```
async def run_scheme_agent(self, audio_input: bytes, user_context: Dict[str, Any]) -> Dict[str, Any]:
```

```
    """
```

```
    Primary execution function. Converts audio to intent, updates state,  
    progresses the graph, and responds via voice.
```

```
    """
```

```
    thread_id = user_context.get("session_id", str(uuid.uuid4()))
```

```
    config = {"configurable": {"thread_id": thread_id}}
```

```
    language = user_context.get("language_pref", "hi-IN")
```

```
# 1. Voice to Text Translation (Sarvam AI)
```

```
user_text = await self.voice_service.speech_to_text(audio_input, language_code=language)
```

```

    if not user_text:
        return {"error": "Audio parsing failed. Please try speaking clearly."}

    logger.info(f"User Input ({thread_id}): {user_text}")

    # 2. Inject Context and Trigger Graph Execution
    input_state = {
        "messages": [HumanMessage(content=user_text)],
        "language_pref": language,
        "user_profile": user_context.get("profile", {})
    }

    # RAG Retrieval is injected if a scheme isn't already targeted
    current_snapshot = await self.app.checkpointer.aget(config)
    if not current_snapshot or not current_snapshot.values.get("target_scheme"):
        user_state = user_context.get("profile", {}).get("state", "Central")
        user_land = user_context.get("profile", {}).get("land_area", 0.0)

        docs = await self.retriever.retrieve_relevant_schemes(user_text, user_state,
float(user_land))
        if docs:
            # Retrieve the JSON rules mapped to the top vector hit
            target_scheme_json = docs.metadata
            input_state["target_scheme"] = target_scheme_json
            input_state["current_step"] = "scheme_identified"

    # Execute the State Machine
    output_state = await self.app.ainvoke(input_state, config=config)

    # 3. Handle Interrupts (Human Validation)
    interrupts = output_state.get("interrupts",)
    if interrupts:
        return {
            "status": "awaiting_user_confirmation",
            "validation_data": interrupts
        }

    # 4. Text to Speech Response
    ai_message = output_state["messages"][-1].content
    audio_response = await self.voice_service.text_to_speech(ai_message,
target_language=language)

    return {

```

```

        "status": output_state.get("current_step", "in_progress"),
        "text_response": ai_message,
        "audio_response_b64": base64.b64encode(audio_response).decode('utf-8') if
audio_response else None
    }

```

Part 9: Code Structure Architecture

To maintain maintainability and deployment scaling across cloud architectures, the module directory hierarchy isolates external service adapters from internal agent logic.

gov_advisory_backend/

```

├── agents/
|   ├── init.py
|   └── scheme_agent.py # State definitions, Nodes, and Routing Logic
├── pipeline/
|   ├── init.py
|   ├── orchestration.py # SchemeOrchestrator, Graph Compilation
|   └── endpoints.py # FastAPI routes for Human-in-the-Loop
├── services/
|   ├── init.py
|   ├── scheme_service.py # Async data scraping & API Setu ingestion
|   ├── retrieval_service.py # pgvector embedding and metadata filtering
|   ├── eligibility_engine.py # Hybrid Rules/LLM validation logic
|   ├── voice_service.py # Sarvam AI STT & TTS integration
|   └── form_filler.py # PyMuPDF dictionary-to-PDF mapping
├── templates/
|   └── PM-KISAN-001_template.pdf
├── outputs/ # Generated flattened PDFs
├── requirements.txt
└── main.py # Uvicorn entry point

```

Conclusion

The architecture presented establishes an enterprise-ready, voice-first ecosystem uniquely suited for agricultural GovTech deployment. By enforcing metadata filtering at the vector retrieval layer and isolating business logic through a Hybrid Eligibility Engine, the system guarantees compliance with policy constraints. Furthermore, deploying LangGraph's persistent state machine seamlessly manages the complexities of interrupted multi-turn data collection, enabling users to generate securely flattened, submission-ready PDF forms entirely via conversational dialect.

Works cited

1. myScheme - Apps on Google Play, accessed on April 26, 2026, https://play.google.com/store/apps/details?id=in.gov.negd.myscheme&hl=en_US
2. State Agriculture Mechanization Scheme - MahaDBT, accessed on April 26, 2026, <https://mahadbt.maharashtra.gov.in/Farmer/SchemeData/SchemeData?str=E9DDFA703C38E51A147B39AD4D6A9082>
3. Pradhan Mantri Kisan Samman Nidhi (PM Kisan) - Digital India | Leading the transformation in India for ease of living and digital economy, accessed on April 26, 2026, <https://www.digitalindia.gov.in/initiative/pm-kisan/>
4. API SETU - Digital India | Leading the transformation in India for ease of living and digital economy | MeitY, Government of India, accessed on April 26, 2026, <https://www.digitalindia.gov.in/initiative/api-setu/>
5. Agri Stack, accessed on April 26, 2026, <https://agristack.gov.in/>
6. API Setu: Get access to thousands of APIs, accessed on April 26, 2026, <https://www.apisetu.gov.in/>
7. Need of retrieval augmented generation for conversational assistants for government social welfare scheme popularization, accessed on April 26, 2026, https://ijsra.net/sites/default/files/fulltext_pdf/IJSRA-2024-1747.pdf
8. PGVector integration - Docs by LangChain, accessed on April 26, 2026, <https://docs.langchain.com/oss/python/integrations/vectorstores/pgvector>
9. PGVectorStore integration - Docs by LangChain, accessed on April 26, 2026, <https://docs.langchain.com/oss/python/integrations/vectorstores/pgvectorstore>
10. Hybrid Validation Agent: Rule-Based Engine + LLM Fallback | by Harsh kumar | Medium, accessed on April 26, 2026, <https://medium.com/@kumarharsh74799/the-hybrid-validation-pattern-rules-first-llm-fallback-cfe545efcd44>
11. Hybrid LLM-Assisted Fault Diagnosis Framework for 5G/6G Networks Using Real-World Logs - MDPI, accessed on April 26, 2026, <https://www.mdpi.com/2073-431X/14/12/551>
12. Hybrid Intelligence: Marrying Deterministic Code with LLMs for Robust Software Development - New Math Data, accessed on April 26, 2026, <https://newmathdata.com/blog/hybrid-ai-deterministic-code-llm-reasoning-systems/>
13. Speech to Text API for Indian Languages - Sarvam AI, accessed on April 26, 2026, <https://www.sarvam.ai/apis/speech-to-text>
14. Text-to-Speech Rest API - Sarvam AI Docs, accessed on April 26, 2026,

<https://docs.sarvam.ai/api-reference-docs/api-guides-tutorials/text-to-speech/rest-api>

15. Getting Started with Sarvam AI APIs Using Python | by Ashieshmittapalli | Medium, accessed on April 26, 2026, <https://medium.com/@ashieshmittapalli/getting-started-with-sarvam-ai-apis-using-python-fc91f111b501>
16. Built with LangGraph! #19: State Machines | by Okan Yenigün | Stackademic, accessed on April 26, 2026, <https://blog.stackademic.com/built-with-langgraph-19-state-machines-24e9c5de8869>
17. LangGraph overview - Docs by LangChain, accessed on April 26, 2026, <https://docs.langchain.com/oss/python/langgraph/overview>
18. LangGraph State Machines: Managing Complex Agent Task Flows in Production, accessed on April 26, 2026, <https://dev.to/jamesli/langgraph-state-machines-managing-complex-agent-task-flows-in-production-36f4>
19. PyMuPDF is a high performance Python library for data extraction, analysis, conversion & manipulation of PDF (and other) documents. - GitHub, accessed on April 26, 2026, <https://github.com/pymupdf/pymupdf>
20. Automating PDF Form Filling and Flattening with PyMuPDF - Artifex Software, Inc, accessed on April 26, 2026, <https://artifex.com/blog/automating-pdf-form-filling-and-flattening-with-pymupdf>
21. Interrupts - Docs by LangChain, accessed on April 26, 2026, <https://docs.langchain.com/oss/python/langgraph/interrupts>
22. Validating human input - while loop example repeated based on number of invocation · Issue #4172 · langchain-ai/langgraph - GitHub, accessed on April 26, 2026, <https://github.com/langchain-ai/langgraph/issues/4172>